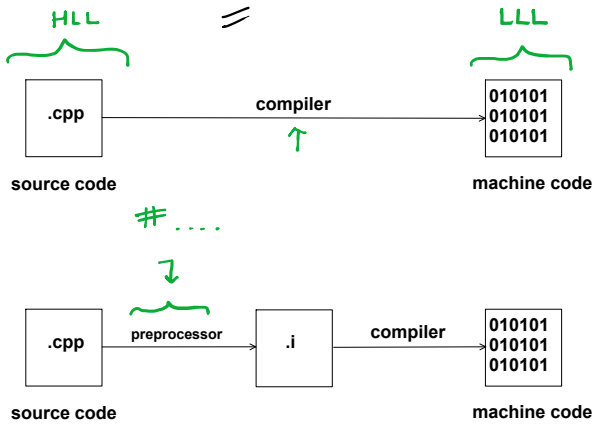


Introduction to C++

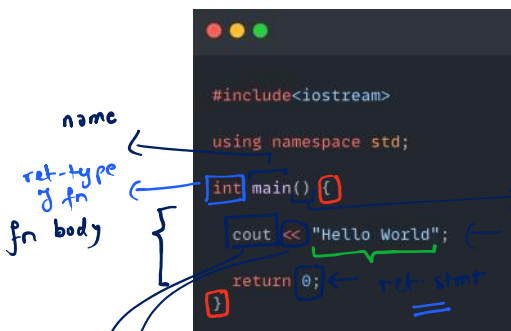
➤ The **extension** of C++ file is **.cpp**



What is a function ?

In C++, a function is a **group of statements** which upon **invocation** or **calling** performs a task. In general, you can compare them with mathematical functions

1. name $\Rightarrow f(x) = x^2$
 2. 0 or more args. $\Rightarrow g(x, y) = x^2 + y^2$
 3. task
 4. a c++ fn can ret 0 or 1 val. once all its stmts. are executed
- $f(2) \rightarrow 4$
 $f(3) \rightarrow 9$
 $\vdots$
 $g(2, 3) \rightarrow 13$
 $g(4, 5) \rightarrow 41$



* As soon as you enter the stmt. w/ the body of the fn, the fn ends.

* a fn can have multiple ret. stmt. As soon as you encounter the 1st one, the fn ends.

`cout <<`
 fn call
 this fn is defined in the iostream header file

`cout << 10;`
`cout << 'x';`
`cout << 3.14;`
`cout << 10+20;`
`cout << true; // 1`
`cout << false; // 0`

```

cout << true; // 1
cout << false; // 0

```

Comments in C++

A **comment** is a **line of text** which is used to **document a program** and is **ignored** by the **compiler**.

```

#include<iostream>
using namespace std;

int main() {
    // this is a single-line comment
    cout << "Hello World;
    return 0;
}

#include<iostream>
using namespace std;

int main() {
    /* this is a multi-line comment
    this is a multi-line comment
    this is a multi-line comment */
    cout << "Hello World;
    return 0;
}

```

Reading Input in C++

1B = 8 bits

fundamental type

int	→	4B (32-bits)	sizeof(int)	4
char	→	1B (8-bits)	sizeof(char)	1
float	→	4B (32-bits)	sizeof(float)	4
double	→	8B (64-bits)	sizeof(double)	8
bool	→	1B (8-bits)	sizeof(bool)	1

String

$(27)_{10} \rightarrow 0 \dots 011011$ (27 times)

$(10)_{10} \rightarrow (1010)_2$

$000 \dots 01010$ (28 times)

type var-name;

syntax for variable declaration

int x; (int) $\boxed{\dots}$ 4B

char ch; (char) $\boxed{\dots}$ 1B

double y; (double) $\boxed{\dots}$ 8B

bool b; (bool) $\boxed{\dots}$ 1B

$\{ \text{int } x; \text{cin} \gg x; \}$

↓ extraction op.

char-
i/p

$\boxed{\text{cin} \gg}$ fn call

this fn is defined in the iostream

int x; ✓

4B i.e 32 bits

$\boxed{x}$ (int)

-2^{32-1} to $2^{32-1}-1$

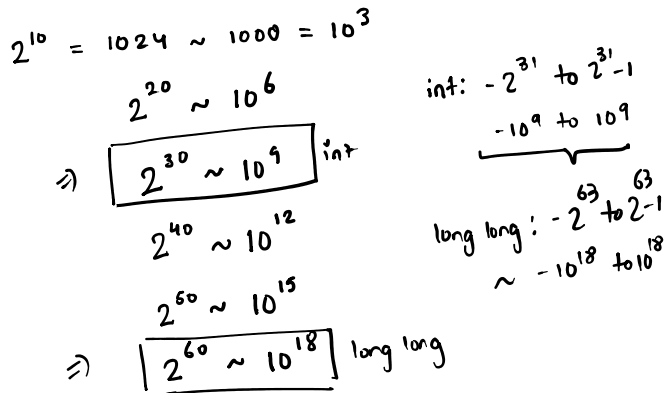
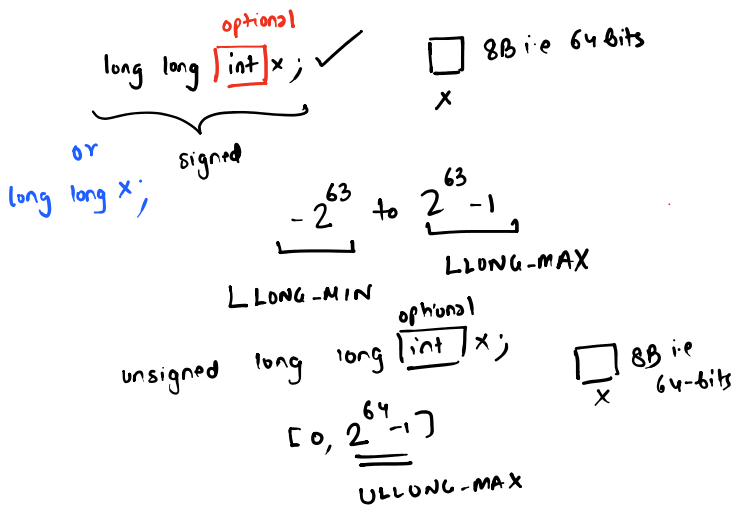
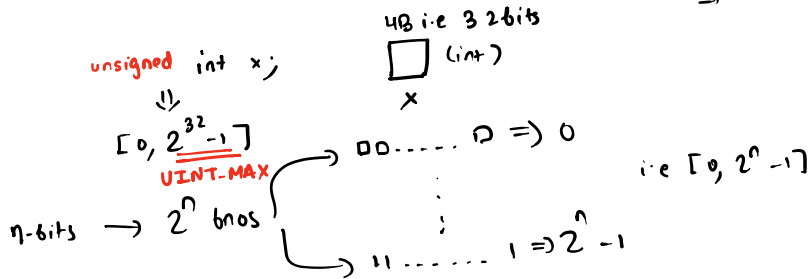
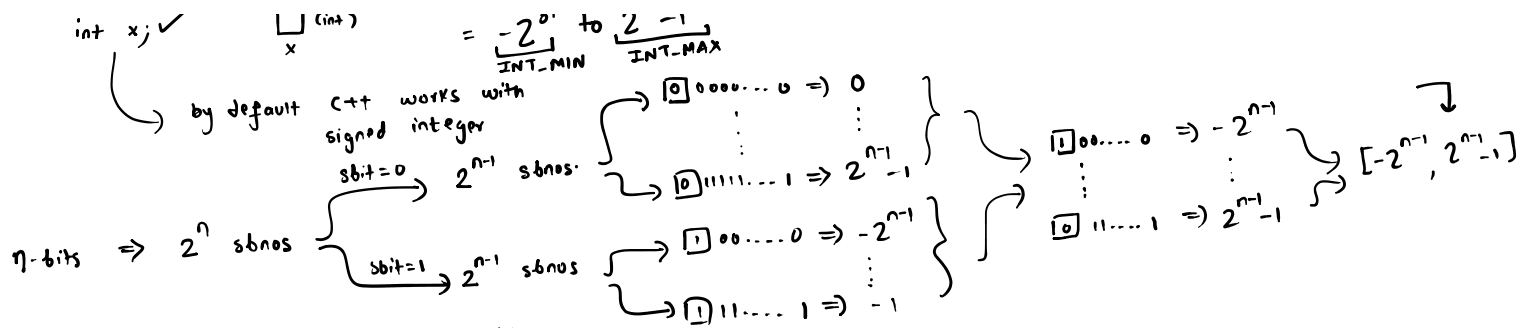
$= -2^{31}$ to $2^{31}-1$

INT_MIN INT_MAX

<limits>

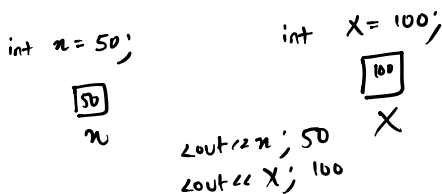
0 0000...0 = 0

Default C++ works with



Naming Rules for Variables

- It must only contain uppercase or lowercase alphabets, digits and underscore.
- It must **not** start with a digit.
- It should not be a **keyword**. e.g. return, main, int, true, false etc.
- In C++, variable names are case-sensitive.



Common Issues using Variables

In C++,

- You cannot **use a variable** before you **declare** it.
- You can create **multiple variables** with the **same names**.
- You cannot **re-declare** a variable.
- By default a variable contains a **garbage** value.

⇒

```
int x = 10;
:
:
char x = 'A'; ← error
:
:
cout << x; // ambiguous

int x;
:
:
int x; // redeclaration / error
```

Arithmetic Operators in C++

⇒ C++ supports five basic arithmetic operations, these include

- Addition (+) : to add two numbers
- Subtraction (-) : to subtract two numbers or to denote a negative number
- Multiplication (*) : to multiply two numbers
- Division (/) : to divide two numbers
- Modulus (%) : returns remainder of dividing the 1st operand by the 2nd

```
int a = 10;
int b = 5;

// addition
cout << a + b << endl; // 15

// subtraction
cout << a - b << endl; // 5

// multiplication
cout << a * b << endl; // 50

// division
cout << a / b << endl; // 2

// modulus
cout << a % b << endl; // 0
```

$\boxed{10}$ $\boxed{5}$
a b

int op int
int

7 / 2
3

in c++,
we do
floor
division

floor []

[3.7] = 3

ceil []

[4.2] = 5

float op int
float

7 / 2.0
3.5

7.0 / 2
3.5

3.5
(float) 7 / 2

explicit type-casting

{ * > + }

Operator Precedence and Associativity

The precedence of an operator dictates the order of operations in an expression.

Example

2 + 3 * 4 = 23

The **precedence** of an operator dictates the **order of operations** in an expression.

Example

$\Rightarrow 3 + 5 * 4 = 23$
 $8 - 4 / 2 = 6$

$8 - 4 / 2 = 6$ (handwritten: $8 - 4 / 2 = 6$ with red brackets under $4/2$)

$3 + 5 * 4 = 23$ (handwritten: $3 + 5 * 4 = 23$ with a bracket under $5 * 4$ labeled 20)

In C++, we can use **parentheses ()** to **enforce** precedence.

$(3 + 5) * 4 = 32$

$\Rightarrow (8 - 4) / 2 = 2$

The **associativity** of an operator is used to determine the order of operations when operators in an expression have the same precedence.

$100 / 10 * 10 = 100$

$5 - 2 + 3 = 6$

left to right associative

$100 / 10 * 10 = 100$ (handwritten: $100 / 10 = 10$, then $10 * 10 = 100$)

$5 - 2 + 3$ (handwritten: $5 - 2 = 3$, then $3 + 3 = 6$)

Simple Interest

$\Rightarrow$ Given the values of **principal**, **rate** and **time**, write a program to find **simple interest**.

int int int

$\leq 10^9$

Relational or Boolean Operators in C++

C++ offers six different **Relational** or **Boolean** operators

Boolean Comparator	Example	Meaning
>	a > b	a greater than b
<	a < b	a less than b
>=	a >= b	a greater than or equal to b
<=	a <= b	a less than or equal to b
==	a == b	a equal to b
!=	a != b	a not equal to b

A **Relational** or **Boolean** operator always evaluates to **True** or **False** .



```
int a = 10;
int b = 5;

cout << ( a > b ); // 1
cout << ( a == b ); // 0
cout << ( a != b ); // 1
```

Logical Operators in C++

C++ offers three different **logical** operators

P	Q	P and Q	P or Q
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

P	not P
True	False
False	True

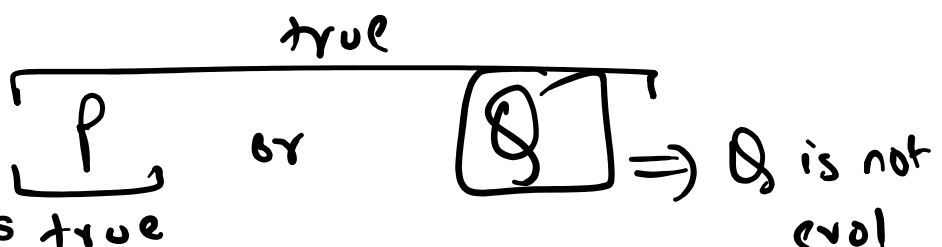
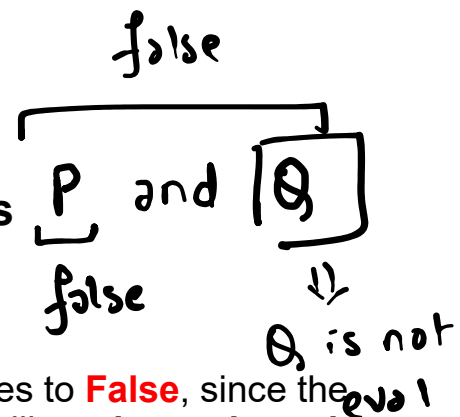
- precedence(not) > precedence(and) > precedence(or)
- All the logical operators are **left to right associative**.



Short-Circuiting of Logical and & or Operators

Consider two Boolean expressions "P" & "Q"

- During evaluation of expression **P and Q**, if "P" evaluates to **False**, since the overall expression evaluates to **False**, expression "Q" will **not be evaluated**.
- During evaluation of expression **P or Q**, if "P" evaluates to **True**, since the overall expression evaluates to **True**, expression "Q" will **not be evaluated**.



Truthy vs Falsy Values

In C++, **individual values** can evaluate to **True** or **False**. The values which evaluate to **True** are considered **Truthy** & values that evaluate to **False** are considered **Falsy**. **Truthy** values include **non-zero** numbers, and the constant **true**. In contrast, **Falsy** values include the **numeric zero-value** and the constants **false** & **NULL**.

0 → false

-ve, +ve → true

```
#include<iostream>

using namespace std;

int main() {
    int x = 10, y = 0;
    cout << (x and y) << endl;
    cout << (x or y) << endl;
    return 0;
}
```

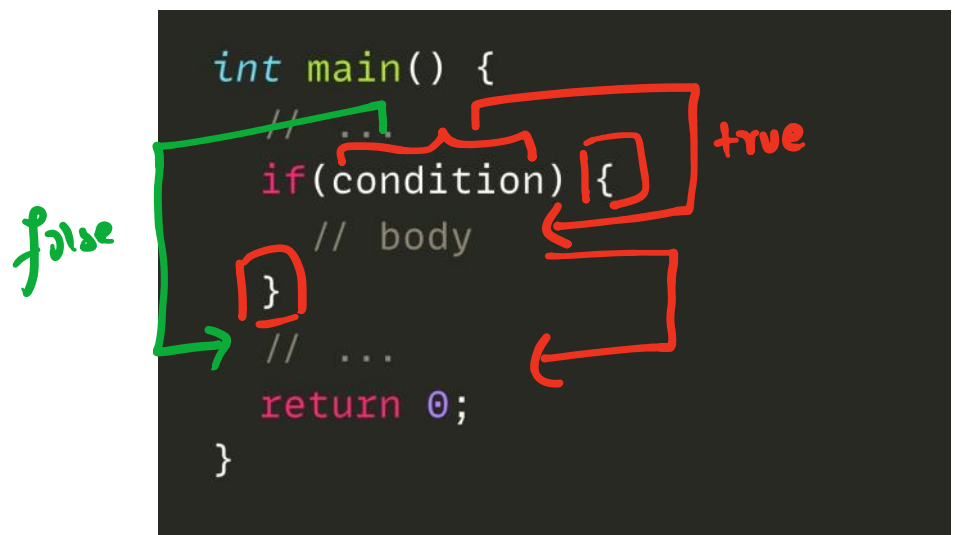
┌

10 and 0
true
false
0

10 or 0
true false
true
1

Conditional Logic in C++

➤ if Statement



➤ else Statement

```

int main() {
    // ...
    if(condition) {
        // body
    } else {
        // body
    }
    // ...
    return 0;
}

```

Diagram illustrating the execution flow of an `if-else` statement. A red bracket labeled *false* points to the `else` block. A green bracket labeled *true* points to the `if` block.

➤ **else if** Statement

Handwritten notes illustrating grade ranges and their corresponding letter grades:

- 91-100 ⇒ 'A'
- 81-90 ⇒ 'B'
- 71-80 ⇒ 'C'
- 61-70 ⇒ 'D'
- 51-60 ⇒ 'E'
- otherwise ⇒ 'F'

```

int main() {
    // ...
    if(condition_1) {
        // ...
    } else if(condition_2) {
        // ...
    } else if(condition_3) {
        // ...
    } else {
        // ...
    }
    // ...
    return 0;
}

```

Diagram illustrating the execution flow of an `else if` statement. A red bracket labeled *optional* points to the `else` block.

Largest of three numbers

Given **three numbers**, design an algorithm to find the **largest** number.

Example

Input : 2, 6, 4

Output : 6

Loops in C++

A **loop** is programming construct that allows you to do the **same task** repeatedly **while** a **condition is true**. In general, a loop has four components

- Initialization
- Condition Checking
- Body of the loop
- Update Rule

➤ The while Loop

```
int main() {  
    // ...  
    // initialization  
    while(condition) {  
        // body of the loop  
    }  
    // ...  
    return 0;  
}
```

Diagram illustrating the while loop structure with annotations:

- Initialization**: Points to the code before the `while` loop.
- Condition Checking**: Points to the `while(condition)` statement.
- Body of the loop**: Points to the code inside the curly braces.
- Update Rule**: Points to the code after the loop body.

Flowchart illustrating the while loop logic:

- init** (Initialization) leads to **cond?** (Condition).
- If **cond?** is **false**, the flow goes to "you go outside the loop".
- If **cond?** is **true**, the flow goes to **body + update** (Body of the loop and Update Rule).
- The flow then loops back to **cond?**.

i/p
n=5

o/p 1 2 3 4 5

Homework

N = 5

```

1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
  
```

N = 5

```

1
0 1
1 0 1
0 1 0 1
1 0 1 0 1
  
```

N = 4

```

1
2 3
4 5 6
7 8 9 10
  
```

N = 5

```

                    1
                  2 3
                3 4 5
              4 5 6 7
            5 6 7 8 9
  
```

N = 5

```

                    1
                  2 3 2
                3 4 5 4 3
              4 5 6 7 6 5 4
            5 6 7 8 9 8 7 6 5
  
```

n = 5

```

* * * * *
*       *
*       *
*       *
*       *
* * * * *
  
```

n = 5

```

*           *
*       *   *
*           *
*       *   *
*           *
  
```

n = 5

```

* * * * * * * * *
*       * * * *
*           * * *
*               *
*                   *
  
```

n = 5

```

                    *
                  *
                *
              *
            *
          *
        *
      *
    *
  *
  
```

n = 5

```

* * * * *
*       *
*           *
*               *
*                   *
  
```